

# CampusEvents

## *Campus Event & Ticket Booking System*

COP 4710 — Database Systems   University of South Florida   Spring 2026

---

### Intermediate Report 1 — Due March 30, 2026

## 1. Team Roster

---

This project is submitted by the two-person team listed below. Per Dr. Tu’s course guidelines, two-person groups are held to the same deliverable expectations as a three-person group, and our system design reflects that scope accordingly.

| Name             | Email                  | Role      |
|------------------|------------------------|-----------|
| Trevor Flahardy  | trevorflahardy@usf.edu | Developer |
| Sofia Cobo Navas | scobonavas@usf.edu     | Developer |

## 2. System Overview

---

CampusEvents is a web-based enterprise information system designed to manage campus events at a university. The platform serves three distinct user roles — **students**, who browse and register for events; **organizers**, who create and manage them; and **administrators**, who oversee the system as a whole. The core value proposition is replacing ad-hoc flyers and email blasts with a centralized, queryable system for event discovery and attendance tracking.

The system follows a standard three-tier architecture. The presentation tier is a single-page application built with React and Vite, styled with Tailwind CSS. The application logic tier is a REST API written in TypeScript running on the Bun runtime with the Hono framework. The data tier is a PostgreSQL 16 relational database, accessed through Drizzle ORM — a lightweight TypeScript-native library that serves as the JDBC-equivalent data access layer.

| Tier         | Technology                        | Notes                                                 |
|--------------|-----------------------------------|-------------------------------------------------------|
| Presentation | React 19 + Vite + Tailwind CSS v4 | Web-based UI; ≥ 3 distinct pages                      |
| Application  | Bun + Hono (REST API, TypeScript) | Application logic; handles routing and business rules |

| Tier | Technology                  | Notes                                                         |
|------|-----------------------------|---------------------------------------------------------------|
| Data | PostgreSQL 16 + Drizzle ORM | Relational DBMS; JDBC-equivalent via <code>postgres.js</code> |

### 3. Conceptual Design

The database is modeled around **five entity sets** connected by **four relationship sets**. The design prioritizes normalization and referential integrity — every relationship is enforced at the schema level via foreign keys and `CHECK` constraints rather than relying on application-layer validation alone.

#### 3.1. ER Diagram

The diagram below represents the entity-relationship model for CampusEvents. Entities are shown as labeled rectangles, and the relationships between them are annotated with their cardinality. A machine-readable version of this diagram is also available as `schema_diagram.mermaid` in the report directory.

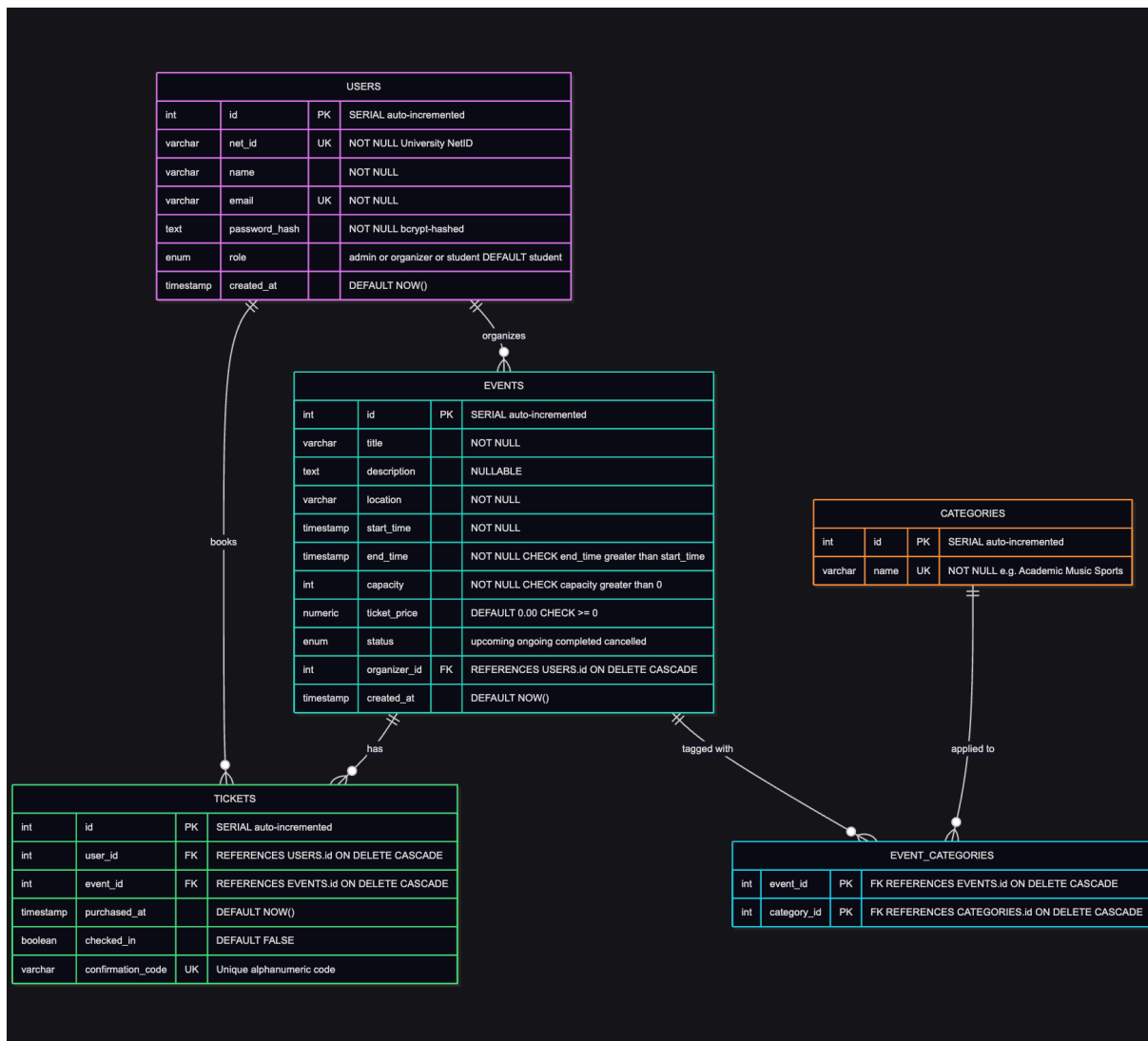


Figure 1: ER Diagram for CampusEvents — showing entity sets, attributes, and relationships with cardinality.

### 3.2. Entity Sets & Attributes

Each entity set is described below with its full attribute list, types, and constraints. Primary keys are underlined by convention; foreign keys are noted explicitly.

**USERS** — Represents every account in the system. The `role` attribute determines what actions a user may perform: `student` accounts can browse and book tickets, `organizer` accounts can create and manage events, and `admin` accounts have full system access.

| Attribute | Type   | Constraint  | Notes                          |
|-----------|--------|-------------|--------------------------------|
| <u>id</u> | SERIAL | PRIMARY KEY | Auto-incremented surrogate key |

| Attribute                  | Type                      | Constraint                     | Notes                                                              |
|----------------------------|---------------------------|--------------------------------|--------------------------------------------------------------------|
| <code>net_id</code>        | <code>VARCHAR(50)</code>  | <code>NOT NULL, UNIQUE</code>  | University-issued NetID (e.g., <code>jsmith22</code> )             |
| <code>name</code>          | <code>VARCHAR(100)</code> | <code>NOT NULL</code>          | Full display name                                                  |
| <code>email</code>         | <code>VARCHAR(150)</code> | <code>NOT NULL, UNIQUE</code>  | University email address                                           |
| <code>password_hash</code> | <code>TEXT</code>         | <code>NOT NULL</code>          | bcrypt-hashed; never stored in plaintext                           |
| <code>role</code>          | <code>ENUM</code>         | <code>DEFAULT 'student'</code> | <code>admin</code>   <code>organizer</code>   <code>student</code> |
| <code>created_at</code>    | <code>TIMESTAMP</code>    | <code>DEFAULT NOW()</code>     | Account creation timestamp                                         |

**EVENTS** — The central entity of the system. Every event has a designated organizer (a foreign key to `USERS`), a time range, a physical location, and a capacity ceiling. The `ticket_price` field defaults to `0.00`, making free events the default case. The `status` enum drives filtering throughout the UI.

| Attribute                 | Type                       | Constraint                             | Notes                                                                                          |
|---------------------------|----------------------------|----------------------------------------|------------------------------------------------------------------------------------------------|
| <code>id</code>           | <code>SERIAL</code>        | <code>PRIMARY KEY</code>               | Auto-incremented surrogate key                                                                 |
| <code>title</code>        | <code>VARCHAR(200)</code>  | <code>NOT NULL</code>                  | Event display name                                                                             |
| <code>description</code>  | <code>TEXT</code>          | <code>NULLABLE</code>                  | Optional long-form description                                                                 |
| <code>location</code>     | <code>VARCHAR(200)</code>  | <code>NOT NULL</code>                  | Venue name or address                                                                          |
| <code>start_time</code>   | <code>TIMESTAMP</code>     | <code>NOT NULL</code>                  | Event start date and time                                                                      |
| <code>end_time</code>     | <code>TIMESTAMP</code>     | <code>NOT NULL, &gt; start_time</code> | <code>CHECK</code> ensures end is after start                                                  |
| <code>capacity</code>     | <code>INTEGER</code>       | <code>CHECK &gt; 0</code>              | Maximum number of attendees                                                                    |
| <code>ticket_price</code> | <code>NUMERIC(10,2)</code> | <code>DEFAULT 0.00, &gt;= 0</code>     | <code>0.00</code> indicates a free event                                                       |
| <code>status</code>       | <code>ENUM</code>          | <code>DEFAULT 'upcoming'</code>        | <code>upcoming</code>   <code>ongoing</code>   <code>completed</code>   <code>cancelled</code> |
| <code>organizer_id</code> | <code>INTEGER</code>       | <code>FK → USERS.id</code>             | References the event creator                                                                   |
| <code>created_at</code>   | <code>TIMESTAMP</code>     | <code>DEFAULT NOW()</code>             | Record insertion timestamp                                                                     |

**TICKETS** — A ticket represents a confirmed booking between a user and an event. The `UNIQUE(user_id, event_id)` constraint enforces the business rule that a student may only hold

one ticket per event. The `confirmation_code` is a short, human-readable identifier shown to the user after booking and used for check-in.

| Attribute                        | Type                     | Constraint                    | Notes                                     |
|----------------------------------|--------------------------|-------------------------------|-------------------------------------------|
| <code>id</code>                  | <code>SERIAL</code>      | <code>PRIMARY KEY</code>      | Auto-incremented surrogate key            |
| <code>user_id</code>             | <code>INTEGER</code>     | <code>FK → USERS.id</code>    | References the ticket holder              |
| <code>event_id</code>            | <code>INTEGER</code>     | <code>FK → EVENTS.id</code>   | References the booked event               |
| <code>purchased_at</code>        | <code>TIMESTAMP</code>   | <code>DEFAULT NOW()</code>    | Booking timestamp                         |
| <code>checked_in</code>          | <code>BOOLEAN</code>     | <code>DEFAULT FALSE</code>    | Set to <code>TRUE</code> upon event entry |
| <code>confirmation_code</code>   | <code>VARCHAR(20)</code> | <code>UNIQUE</code>           | Short alphanumeric booking code           |
| <code>(user_id, event_id)</code> | —                        | <code>UNIQUE composite</code> | Prevents duplicate bookings               |

**CATEGORIES** and **EVENT\_CATEGORIES** — Categories form a simple lookup table (e.g., Academic, Music, Sports, Career, Technology). The many-to-many relationship between **EVENTS** and **CATEGORIES** is resolved through the **EVENT\_CATEGORIES** join table, allowing a single event to belong to multiple categories for filtering purposes.

| Attribute                | Type                      | Constant                         |
|--------------------------|---------------------------|----------------------------------|
| <code>id</code>          | <code>SERIAL</code>       | <code>PRIMARY KEY</code>         |
| <code>name</code>        | <code>VARCHAR(100)</code> | <code>NOT NULL, UNIQUE</code>    |
| <code>event_id</code>    | <code>INTEGER</code>      | <code>FK → EVENTS, PK</code>     |
| <code>category_id</code> | <code>INTEGER</code>      | <code>FK → CATEGORIES, PK</code> |

## 4. Relationship Sets

The four relationship sets in the model are summarized below. Each is enforced at the schema level — cardinality is not merely documented but implemented through foreign key constraints and, in the case of the many-to-many relationship, a dedicated join table.

| Entity A | Entity B   | Name               | Cardinality | Description                                                                  |
|----------|------------|--------------------|-------------|------------------------------------------------------------------------------|
| USERS    | EVENTS     | <i>organizes</i>   | 1 : N       | An organizer may create many events; each event has exactly one organizer    |
| USERS    | TICKETS    | <i>books</i>       | 1 : N       | A student may hold tickets to many events; each ticket belongs to one user   |
| EVENTS   | TICKETS    | <i>has</i>         | 1 : N       | An event may have many tickets; each ticket corresponds to exactly one event |
| EVENTS   | CATEGORIES | <i>tagged with</i> | M : N       | Resolved via EVENT_CATEGORIES; an event may belong to multiple categories    |

## 5. Key Constraints

Beyond standard foreign key and uniqueness constraints, the schema enforces the following business rules at the database level:

- `end_time > start_time` — A `CHECK` constraint on `EVENTS` ensures that no event can be created with an end time that precedes or equals its start time.
- `capacity > 0` — A `CHECK` constraint ensures every event must allow at least one attendee.
- `ticket_price >= 0` — Prevents negative ticket prices from being inserted.
- `UNIQUE(user_id, event_id)` on `TICKETS` — Enforces the one-ticket-per-user-per-event rule at the schema level, making it impossible to double-book regardless of application behavior.
- `confirmation_code UNIQUE` — Every ticket carries a globally unique confirmation code for check-in purposes.
- `net_id UNIQUE` — Prevents duplicate accounts sharing the same university NetID.
- `ON DELETE CASCADE` on `TICKETS` — Deleting a user or event automatically removes all associated tickets and category links, maintaining referential integrity without manual cleanup.

The SQL `CREATE TABLE` statements encoding these constraints are included in `schema.sql` at the project root. A representative excerpt is shown below:

```
CREATE TABLE events (
  id          SERIAL          PRIMARY KEY,
  title       VARCHAR(200)    NOT NULL,
  location    VARCHAR(200)    NOT NULL,
  start_time  TIMESTAMP       NOT NULL,
  end_time    TIMESTAMP       NOT NULL,
  capacity    INTEGER          NOT NULL CHECK (capacity > 0),
  ticket_price NUMERIC(10, 2) NOT NULL DEFAULT 0.00
              CHECK (ticket_price >= 0),
  status      event_status    NOT NULL DEFAULT 'upcoming',
```

```

    organizer_id INTEGER NOT NULL REFERENCES users(id)
                      ON DELETE CASCADE,

    CONSTRAINT valid_time_range CHECK (end_time > start_time)
);

```

## 6. Planned SQL Query Types

The system is required to support at least eight distinct SQL query types, including queries that modify database state. We have planned ten query types to ensure the requirement is met with margin. Each query corresponds to a core application feature — they are not academic exercises but the actual queries that will back the UI. All ten are defined and commented in `schema.sql`.

| #   | Type                      | Application Feature                                            |
|-----|---------------------------|----------------------------------------------------------------|
| Q1  | SELECT + JOIN             | Browse page — list upcoming events with organizer name         |
| Q2  | SELECT + GROUP BY + COUNT | Dashboard — count tickets sold per event                       |
| Q3  | SELECT + subquery         | Event detail page — compute remaining capacity                 |
| Q4  | SELECT + multiple JOINs   | My Tickets page — show booking history with category tags      |
| Q5  | INSERT                    | Ticket purchase — create a new booking record                  |
| Q6  | UPDATE                    | Check-in flow — set <code>checked_in = TRUE</code> on a ticket |
| Q7  | UPDATE                    | Organizer dashboard — cancel an event by updating its status   |
| Q8  | DELETE                    | Cancel booking — remove a ticket record                        |
| Q9  | SELECT + HAVING           | Analytics — find categories with more than one active event    |
| Q10 | SELECT + BETWEEN          | Date filtering — retrieve events within a specified date range |

A representative sample of how these will be expressed in SQL:

```

-- Q1: Upcoming events with organizer name
SELECT e.title, e.location, e.start_time, u.name AS organizer
FROM events e
JOIN users u ON e.organizer_id = u.id
WHERE e.status = 'upcoming'
ORDER BY e.start_time;

```

```
-- Q3: Remaining capacity via subquery
SELECT e.title,
       e.capacity - (
         SELECT COUNT(*) FROM tickets t WHERE t.event_id = e.id
       ) AS spots_remaining
FROM events e
WHERE e.status = 'upcoming';

-- Q5: Ticket purchase (INSERT)
INSERT INTO tickets (user_id, event_id, confirmation_code)
VALUES ($1, $2, $3);
```

## 7. Planned Interface Pages

The interface requirement specifies at least three distinct pages. We have designed five, three of which fulfill the requirement and two of which are planned bonus features that expand the system's functionality toward a realistic enterprise application.

| Page                 | Access    | Description                                                                                                   |
|----------------------|-----------|---------------------------------------------------------------------------------------------------------------|
| Home / Browse Events | Public    | Lists all upcoming events with search and category filtering. The primary discovery surface for students.     |
| Event Detail         | Public    | Displays full event info, real-time remaining capacity, and a Book Ticket button for authenticated users.     |
| My Tickets           | Student   | Shows the authenticated user's booking history, confirmation codes, and check-in status for each event.       |
| Organizer Dashboard  | Organizer | <b>(Bonus)</b> Allows organizers to create, edit, and cancel events, and view a full attendee list per event. |
| Admin Panel          | Admin     | <b>(Bonus)</b> Full system view — manage all users, events, and categories; assign and revoke user roles.     |

The page structure maps directly to the query plan in Section 6. The Browse Events page is backed by Q1 and Q10; the Event Detail page uses Q3 for capacity calculation; My Tickets uses Q4; the Organizer Dashboard uses Q2 for sales metrics, Q6 for check-in, and Q7 for cancellations; and ticket purchase (Q5) and cancellation (Q8) are triggered from both the detail page and the dashboard.